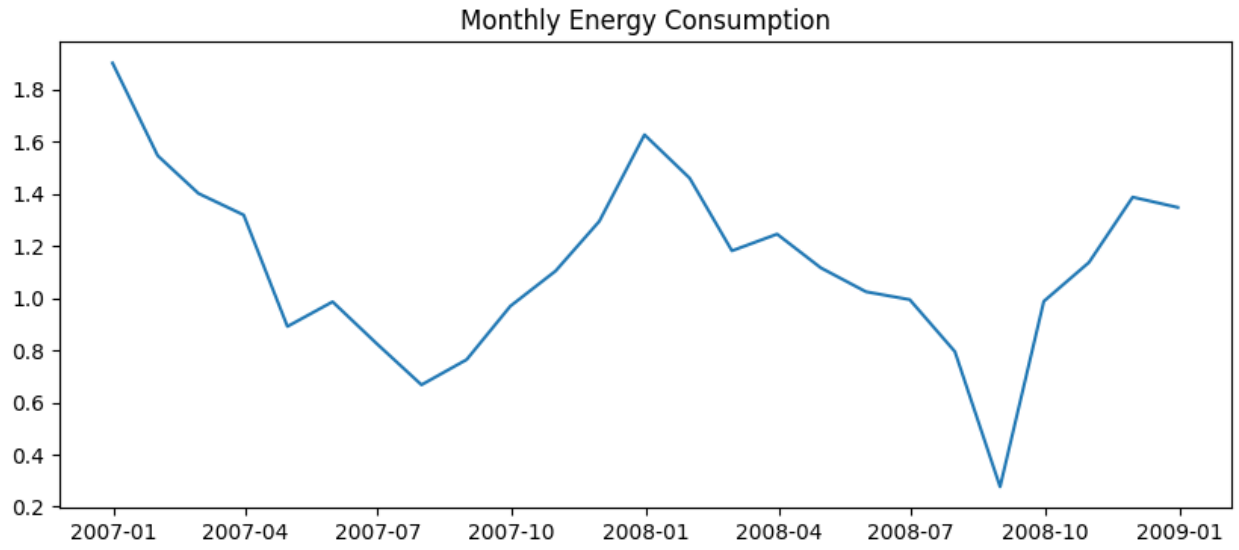
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error
import warnings
warnings.filterwarnings("ignore")

df = pd.read_csv(r"C:\Users\Admin-PC\Downloads\
household_power_consumption.csv\household_power_consumption.csv")

df['Date'] = pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True)

df['Global_active_power'] = pd.to_numeric(df['Global_active_power'],
errors='coerce')

df = df[['Global_active_power']]
df = df.resample('M').mean()

df.rename(columns={'Global_active_power':'Energy'}, inplace=True)

df.head()
```

Date	Energy
2006-12-31	1.901295
2007-01-31	1.546034
2007-02-28	1.401084
2007-03-31	1.318627
2007-04-30	0.891189

```
df['Temperature'] = 30 + 5*np.sin(np.arange(len(df))/12) +
np.random.randint(-2,2,len(df))
df.head()
```

Date	Energy	Temperature
2006-12-31	1.901295	31.000000
2007-01-31	1.546034	31.416185
2007-02-28	1.401084	30.829481
2007-03-31	1.318627	29.237020
2007-04-30	0.891189	30.635973

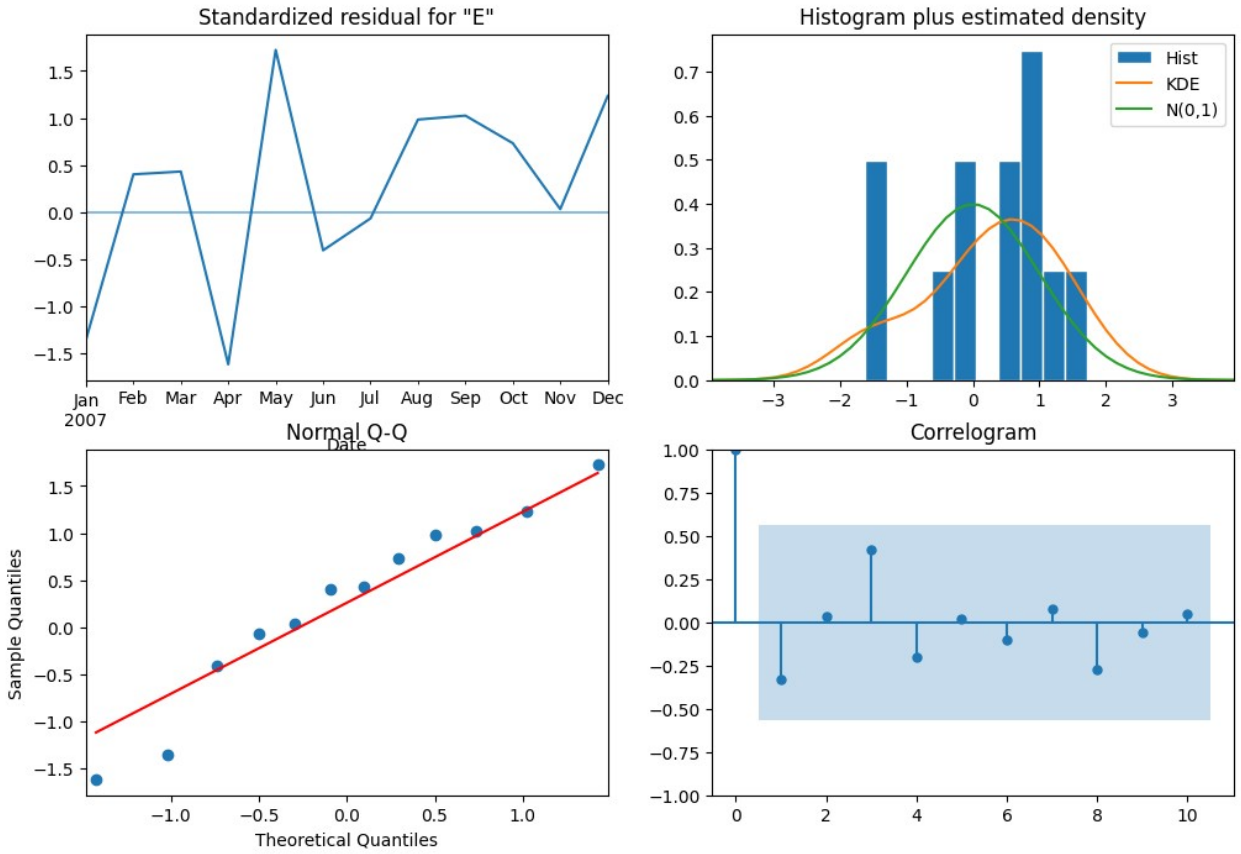
```
plt.figure(figsize=(10,4))
plt.plot(df['Energy'])
plt.title("Monthly Energy Consumption")
plt.show()
```



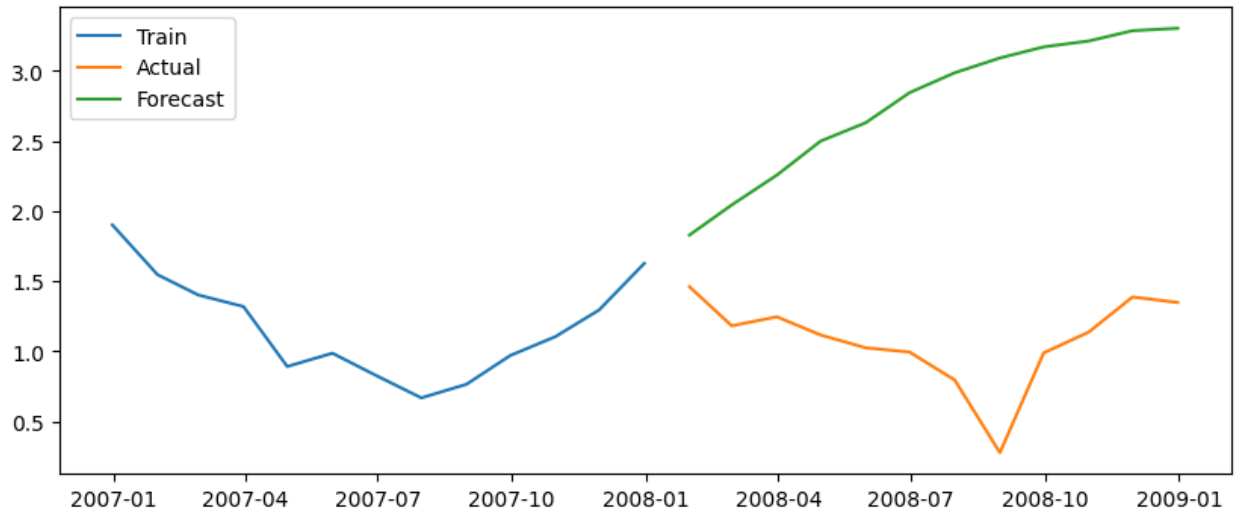

```
train = df.iloc[:-12]
test = df.iloc[-12:]

y_train = train['Energy']
y_test = test['Energy']

exog_train = train[['Temperature']]
exog_test = test[['Temperature']]

sarima = SARIMAX(y_train,
                 order=(1,1,1),
                 seasonal_order=(1,0,1,6)
)

sarima_fit = sarima.fit()
sarima_fit.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
"""
                                     SARIMAX Results

==========================================================================
==================
Dep. Variable:                             Energy   No. Observations:
13
Model:             SARIMAX(1, 1, 1)x(1, 0, 1, 6)   Log Likelihood
2.630
Date:                           Sat, 07 Feb 2026   AIC
4.740
Time:                                   20:58:34   BIC
7.165
Sample:                                12-31-2006   HQIC
3.843
```

```
                                      - 12-31-2007
Covariance Type:                              opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.8685      0.450      1.929      0.054      -0.014       1.751
ma.L1         -0.4668      0.860     -0.543      0.587      -2.153       1.219
ar.S.L6       -0.4632     12.316     -0.038      0.970     -24.603      23.677
ma.S.L6        0.9843    289.704      0.003      0.997    -566.825     568.794
sigma2         0.0271      7.538      0.004      0.997     -14.746      14.801
===================================================================================
Ljung-Box (L1) (Q):                   1.65   Jarque-Bera (JB):                 0.80
Prob(Q):                              0.20   Prob(JB):                         0.67
Heteroskedasticity (H):               0.64   Skew:                            -0.57
Prob(H) (two-sided):                  0.68   Kurtosis:                         2.45
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
"""

sarima_fit.plot_diagnostics(figsize=(12,8))
plt.show()
```




```
forecast = sarima_fit.predict(
    start=test.index[0],
    end=test.index[-1],
    exog=exog_test
)

plt.figure(figsize=(10,4))
plt.plot(train['Energy'], label='Train')
plt.plot(test['Energy'], label='Actual')
plt.plot(forecast, label='Forecast')
plt.legend()
plt.show()
```




```
mae = mean_absolute_error(y_test, forecast)
rmse = np.sqrt(mean_squared_error(y_test, forecast))

print("MAE:", mae)
print("RMSE:", rmse)
```

```
MAE: 1.6834286536740828
RMSE: 1.8039106160681262
```

```
future_temp = pd.DataFrame({'Temperature':[30,31,32,33,32,31]})

future = sarima_fit.forecast(steps=6, exog=future_temp)

future
```

```
2008-01-31    1.828764
2008-02-29    2.043378
2008-03-31    2.256522
2008-04-30    2.498792
2008-05-31    2.630138
2008-06-30    2.844136
Freq: ME, Name: predicted_mean, dtype: float64
```

```
print("SARIMAX captured seasonal energy trends and temperature
influence.")
print("Model supports efficient power distribution planning.")
```

```
SARIMAX captured seasonal energy trends and temperature influence.
Model supports efficient power distribution planning.
```